Université de Haute-Alsace
École Nationale Supérieure d'Ingénieurs
Sud Alsace

Informatique et Réseaux - 2A

Artificial Intelligence and Reinforcement Learning

TD2 : Heuristic Search

Exercise 1

— Path Costs

Figure 1 shows a Pokémon environment where each terrain has a different movement cost.



Figure 1: Pokémon environment with weighted terrain.

The trainer starts at the blue highlighted cell and must reach the Pokémon Center located at the red highlighted cell.

The movement costs are given below.

Terrain	Movement cost
Grass	1
Rock	5
Tree	Blocked (cannot be crossed)

The cost of a path is defined as the sum of the movement costs of all visited terrain cells after the initial state. The initial state has cost 0.

Consider the following two paths shown on the map.

1. Compute the total cost of the path that follows the upper corridor through the rocks.
2. Compute the total cost of the path that follows the lower grassy corridor.
3. Which path has the lowest total cost?
4. Explain why minimizing the number of actions is not necessarily equivalent to minimizing the total path cost.

5. Based on your answer, explain why Breadth-First Search is no longer guaranteed to return an optimal solution when actions have different costs.

Exercise 2
— Dijkstra's Algorithm

We consider the weighted Pokémon environment shown in Figure 1.

The trainer must reach the Pokémon Center while minimizing the total movement cost. Recall that:

Terrain	Movement cost
Grass	1
Rock	5
Tree	Blocked

The cost of moving to a new state is the cost of the terrain entered. The initial state has accumulated cost 0.

To solve this problem, we use Dijkstra's algorithm, also called Uniform-Cost Search in Artificial Intelligence.

The frontier is a priority queue ordered by the accumulated path cost

$$g(s).$$

Use the following rules:

- actions are considered in the order

$$\text{up}, \text{right}, \text{down}, \text{left};$$

- the state with the smallest accumulated cost is removed first;
- when two states have the same cost, the state inserted first is removed first;
- if a cheaper path to a state is found, its stored cost and parent are updated;
- the goal test is performed when a state is removed from the priority queue.

1. Apply Dijkstra's algorithm from the initial state until the Pokémon Center is reached.

   For each removed state, record its accumulated cost.
2. Give the order in which states are removed from the priority queue.
3. Give the path returned by the algorithm as a sequence of states.
4. Give the corresponding sequence of actions.
5. Compute the total cost of the returned path.
6. How many states are removed from the priority queue before the goal is reached, including the goal state?
7. Compare the path returned by Dijkstra's algorithm with the upper rocky path studied in Exercise 1.

   Explain why Dijkstra's algorithm prefers the returned path.

8. Draw the path returned by Dijkstra's algorithm on a copy of Figure 1.

Exercise 3
— A* Search

We now consider the larger Pokémon environment shown in Figure 2.



Figure 2: Larger weighted Pokémon environment used for A* search.

As before, the movement costs are:

Terrain	Movement cost
Grass	1
Rock	5
Tree	Blocked

Rows are numbered from top to bottom and columns from left to right, starting from 1. The initial state is therefore $(2, 2)$, and the goal state is $(11, 17)$.

Dijkstra's algorithm orders the priority queue using only the accumulated path cost

$$g(s),$$

where $g(s)$ is the cost of the cheapest path currently known from the initial state to s.
A* adds a second function, called the **heuristic function**:

$$h(s).$$

This function estimates the remaining cost from state s to the goal. A* orders its priority queue using

$$f(s) = g(s) + h(s).$$

For this exercise, use the Manhattan-distance heuristic

$$h(s) = |r_s - r_g| + |c_s - c_g|\,,$$

where (r_s, c_s) is the current state and (r_g, c_g) is the goal state.

The heuristic assumes that reaching the goal requires at least one unit of cost for each remaining horizontal or vertical movement. It ignores trees and rocks and therefore provides an estimate rather than the exact remaining cost.

Use the following rules:

- actions are considered in the order

$$\text{up}, \text{right}, \text{down}, \text{left};$$

- the state with the smallest value of $f(s)$ is removed first;
- when two states have the same value of $f(s)$, the state inserted first is removed first;
- if a cheaper path to a state is found, its values and parent are updated;
- the goal test is performed when a state is removed from the priority queue;
- a state is counted as considered when it is removed from the priority queue.

1. Compute $h(s)$ and $f(s)$ for the initial state.
2. Give the path returned by A* as a sequence of states.
3. Compute the total cost of the returned path.
4. How many states are considered by A*, including the goal state?
5. Give one example of a state that is not considered by A* before reaching the goal, but that Dijkstra's algorithm would have to consider.

   For this state, compute $g(s)$, $h(s)$, and $f(s)$, and explain how the added heuristic term prevents A* from selecting it before the goal.
6. Briefly explain the role of the heuristic function in A*.

Exercise 4
— Programming Dijkstra and A*

In this exercise, you will extend the Pokémon navigation program developed in TD1.

The environment now contains terrain with different movement costs:

Terrain	Movement cost
Grass	1
Rock	5
Tree	Blocked

The provided program [1] generates a large random map. The initial state and the Pokémon Center are also selected randomly, subject to the following conditions:

[1] https://hadifawaz1999.github.io/Artificial-Intelligence-Reinforcement-Learning--2A-Info/TD/TD_02_heuristic_search/exo4.zip

- the initial state and the goal state are connected;
- they are sufficiently far apart;
- the map contains several alternative routes;
- some routes cross expensive rock cells.

The project contains:

- `pokemon_game.py`: map generation, movement costs, successor generation, path validation, and visualization;
- `search.py`: search algorithms;
- `main.py`: generation of a random map and comparison of the algorithms;
- `requirements.txt`: required Python packages.

Your tasks

Implement the following functions in `search.py`:

- `dijkstra(...)`
- `a_star(...)`

Both functions must return a path represented as a list of states:

$$[\text{initial state}, \ldots, \text{goal state}].$$

For A*, use the Manhattan-distance heuristic

$$h(s) = |r_s - r_g| + |c_s - c_g|,$$

where $s = (r_s, c_s)$ and the goal is (r_g, c_g).

The program must display, for each algorithm:

- the returned path;
- the total path cost;
- the number of considered states;
- an animation of the trainer following the returned path.